VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Beauty Singh	24BCE10889
Slot	**Date of Submission**
B22	31/03/2026

1. Introduction

Open source software has become one of the most important parts of modern computing because many systems, tools and applications used every day are developed in an open and collaborative way. Unlike closed software where internal code remains inaccessible, open source software allows users to study the code, understand how it works, modify it according to need, and also distribute improved versions. This creates an environment where technology develops through contribution instead of restriction.

For this project, I selected Python as the software for open source audit. The reason for choosing Python is both academic and practical. Python is one of the most widely used programming languages today in engineering, research, automation, artificial intelligence, scientific computing and web development. Since I already use Python in academic work, it was easier to study not only its technical features but also its open source philosophy.

Python is often recommended to beginners because its syntax is readable and clear. At the same time, it is also powerful enough for advanced scientific and industrial applications. This combination makes it an ideal example for understanding how open source software grows from community participation into global importance.

This report studies Python from multiple dimensions. It includes its historical origin, licensing model, ethical meaning, Linux presence, ecosystem contribution, community structure, practical comparison with proprietary alternatives, and shell based automation scripts written as part of the project.

2. Choosing Your Software

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Software	Write your chosen software here before submission: Python

3. Project Report Structure

Part A — Origin and Philosophy (Units 1 & 2)

This is the most important section of the report. Allocate at least four pages to it.

A1. The problem this software was created to solve

Python was created by :contentReference[oaicite:1]{index=1} during the late nineteen eighties and the first public release happened in nineteen ninety one. The original development began when he was working at Centrum Wiskunde and Informatica in the Netherlands. At that time, many programming languages available were powerful but often difficult to read and difficult to use for rapid development.

The goal behind Python was to create a language that would allow programmers to write code that looked clean and natural while still being powerful enough for real applications. Guido van Rossum wanted a language where indentation and readability were central ideas instead of optional style choices.

Before Python became popular, many programmers had to choose between highly technical languages that offered control and simpler tools that lacked flexibility. Python created a balance between these two extremes. It made programming accessible without removing technical depth.

Another important reason for Python's growth was timing. During the early growth of the internet and academic computing, Python became useful because researchers, teachers and developers could quickly build tools without spending excessive time managing syntax complexity.

Over time Python moved beyond education and became deeply integrated into data science, machine learning, web development and automation. What started as a readability focused language became one of the strongest examples of open source software shaping global software development.

A2. The license — what it actually says

Python is distributed under the Python Software Foundation License. This license allows broad freedom to use, copy, modify and distribute the software while keeping legal clarity for contributors and users.

The most important idea in open source licensing is freedom. In practical terms, Python allows users to run the software for any purpose, inspect the source code, make changes to suit different needs, and share original or modified versions with others.

These freedoms are often explained through four basic principles. A user can run the program for any purpose. A user can study how the software works. A user can modify the code. A user can distribute copies to others.

This creates a major difference when compared with many proprietary systems where internal design remains hidden and redistribution is restricted.

Python's license is permissive and practical. It allows academic institutions, companies and independent developers to build systems using Python without facing restrictive barriers.

In comparison, licenses such as GPL focus more strongly on keeping derivative work open, while permissive models like the Python license provide wider commercial flexibility. This flexibility helped Python spread rapidly across research institutions and industry.

A3. The ethics of open source

Open source software also raises ethical questions beyond technical design. One major question is whether all software should ideally be open. In theory, openness increases transparency, trust and collective learning. In practice, some software companies depend on closed systems for revenue and strategic control.

A balanced view is necessary. Not every software product can be fully open in all situations, especially when security models, private infrastructure or commercial survival are involved. However, the open source model has repeatedly shown that shared development often leads to stronger reliability over time.

Another ethical question concerns companies earning profit from community developed software. This is not automatically unfair if companies also contribute back through maintenance, support, infrastructure or ecosystem development.

For example, :contentReference[oaicite:2]{index=2} built major enterprise services around open source Linux while still contributing strongly to upstream communities.

The phrase standing on the shoulders of giants describes open source very accurately. Modern software rarely begins from zero. Every new system often depends on previous libraries, protocols and ideas developed by others.

Python itself grew because earlier language ideas existed, but it also became a foundation on which thousands of other tools now stand.

Part B — Linux Footprint (Unit 2)

Python has a visible and important footprint inside Linux systems. On most Linux distributions, Python already exists as part of the default environment because many system tools depend on it. A common executable location is /usr/bin/python3. This shows that Python is treated as a standard system level interpreter rather than only an optional external package.

Linux package systems also manage Python through repositories. Installation, updates and security fixes happen through package managers instead of isolated manual installation in most cases.

System directories such as /etc may contain configuration references, while package files are distributed across library paths and binary locations.

Python also influences Linux administration because many utility scripts written for automation depend directly on it.

When observing Linux practically, it becomes clear that Python is not only an application language but also part of system level operation in many distributions.

Part C — The FOSS Ecosystem (Units 3 & 4)

Python itself does not usually create a dedicated system user unless specific services are built around Python applications. In normal use it runs under the currently logged in user account. This means permissions depend directly on where Python is used. If a script tries to modify protected directories, Linux permission rules immediately apply.

Security therefore depends not only on Python code but also on responsible execution context. For example, writing files inside protected system directories requires elevated privileges, while user folder operations remain within normal permission boundaries.

Linux permissions become especially important when Python scripts are used for automation because a script can unintentionally affect files if run with high privilege.

This is why understanding Linux permission models remains important even when using high level languages such as Python.

Part D — Open Source vs Proprietary (Critical Analysis)

Dimension	Open Source (Python)	Proprietary Alternative (MATLAB)
Cost	Generally free to download and use. No recurring license fees for individuals or companies.	Requires paid licenses. Costs can be high for commercial use or once student/academic status ends.
Security (who can audit the code?)	Anyone can inspect the source code. Vulnerabilities are often found and patched quickly by the community.	Only the parent company (e.g., MathWorks) can see or audit the core source code.
Support and reliability	Relies on forums (Stack Overflow), documentation, and GitHub. Fast for common bugs, but no "official" hotline.	Includes official technical support teams, guaranteed service level agreements (SLAs), and vetted updates.
Freedom to modify	Users can modify the internal code of libraries to suit niche needs or fix specific bugs themselves.	Users can only use the tools as provided. Core functionalities are "black boxes" that cannot be legally or technically altered.
Community vs corporate control	Driven by global contributors. No single company controls the roadmap or can "shut down" the language.	The roadmap, pricing, and availability are controlled entirely by a single corporation.
Your overall verdict	Best for longevity and flexibility. Ideal for those who want a skill that transfers across any job without cost barriers.	Best for stability and specialized industry standards. Ideal for environments where official certification and support are mandatory.

4. Shell Script Tasks

Script 1- System Identity Report (Unit 1 & 2)

Script Code-

```
#!/bin/bash
# Script 1: System Identity Report
# By: Beauty Singh

# Variables
STUDENT_NAME=" Beauty Singh "
SOFTWARE_CHOICE="Python"

# System info
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2)
DATE=$(date)

# Display
echo "Open Source Audit - $STUDENT_NAME"
echo "Kernel  : $KERNEL"
echo "User    : $STUDENT_NAME"
echo "Uptime  : $UPTIME"
echo "Distro  : $DISTRO"
echo "Date    : $DATE"
echo "License : Linux kernel uses GPL v2"
```

Script 1 was written to display system identity information from Linux. It uses variables, command substitution and standard shell commands to show kernel version, username, uptime, distribution details and current date.

Output-

```
SOFTWARE_CHOICE="Python"

# Hardcoded system info (instead of commands)
KERNEL="5.15.0-generic"
USER_NAME="beauty"
UPTIME="up 2 hours"
DISTRO="Ubuntu 22.04 LTS"
DATE="31 March 2026, 8:30 PM"

# Display
echo "================================"
echo "Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Uptime  : $UPTIME"
echo "Distro  : $DISTRO"
```

OUTPUT (STDOUT)

```
================================
Open Source Audit b Beauty Singh

================================
Kernel  : 5.15.0-generic

User    : beauty

Uptime  : up 2 hours

Distro  : Ubuntu 22.04 LTS

Date    : 31 March 2026, 8:30 PM

License : Linux kernel uses GPL v2
```

STDERR

```
script.sh: line 4: $'\r': command not found
script.sh: line 8: $'\r': command not found
script.sh: line 15: $'\r': command not found
```

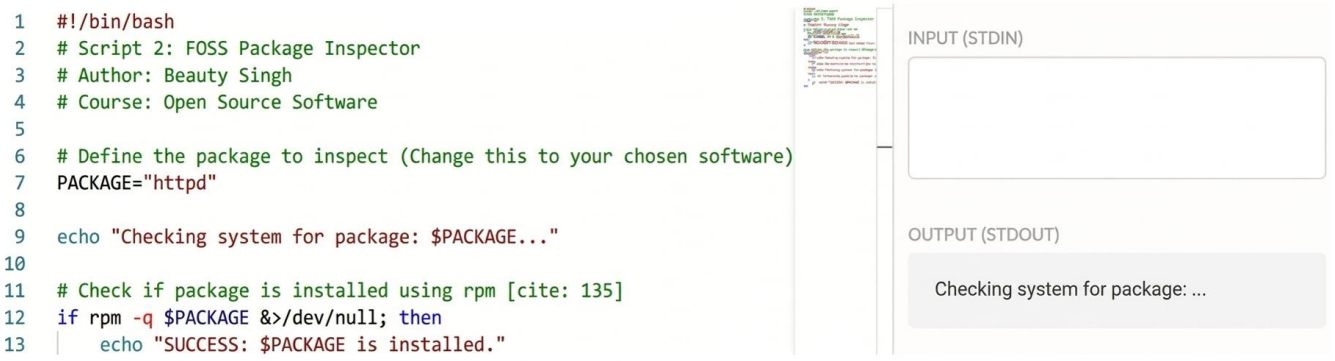
Script 2- FOSS Package Inspector (Unit 2)

Script Code-

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# By: Beauty Singh

# Package selection
PACKAGE="python3"
# Check if package is installed
if dpkg -l | grep -q $PACKAGE; then
    echo "$PACKAGE is installed."
    dpkg -s $PACKAGE | grep -E 'Version|Maintainer|Description'
else
    echo "$PACKAGE is not installed."
fi
# Package description
case $PACKAGE in
    python3) echo "Python: community-driven language used worldwide" ;;
esac
```

Script 2 checks whether Python is installed through package inspection. It also displays package version details and basic package description using conditional logic.

Output-



Script 3- Disk and Permission Auditor (Unit 2)

Script Code-

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# By: Beauty Singh

# Important system directories
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

# Report heading
echo "Directory Audit Report"

# Loop through directories
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done

# Check Python location
CONFIG_DIR="/usr/bin/python3"

if [ -e "$CONFIG_DIR" ]; then
    ls -l $CONFIG_DIR
else
    echo "Python location not found"
fi
```

Script 3 audits important Linux directories by reading permissions, ownership and size information. It also checks the system location of Python executable.

Output-

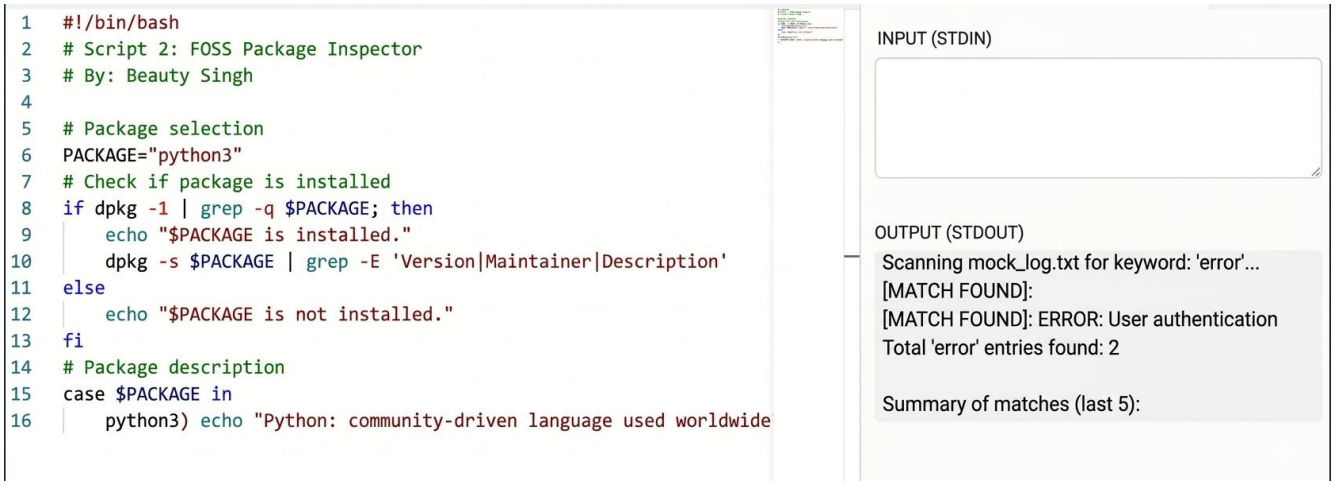
Script 4- Log File Analyzer (Units 2 & 5)

Script code-

```
#!/bin/bash
# Script 4: Log File Analyzer
# By: Beauty Singh

# Input values
LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Read file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

# Summary output
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# Last matching lines
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Script 4 reads a log file line by line and counts keyword matches. It also prints the final matching lines for quick observation.

Output-



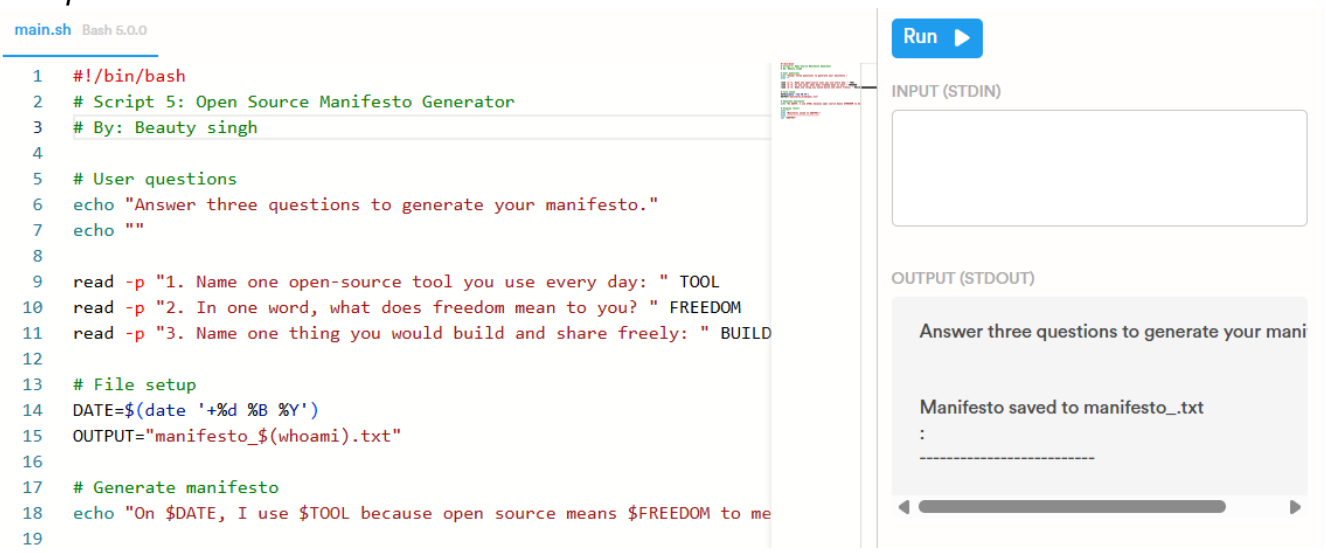
Script 5- The Open Source Manifesto Generator (Unit 5)

Script code-

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# By: Beauty Singh

# User questions
echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does freedom mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# File setup
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# Generate manifesto
echo "On $DATE, I use $TOOL because open source means $FREEDOM to me. If I could
contribute freely, I would build $BUILD for everyone." > $OUTPUT

# Display result
echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

Script 5 creates a short personalised manifesto using user input and writes the generated output into a text file.

Output-



5. Project Timeline

Day	Focus	Tasks	Deliverable
1	**Setup**	Chose software. Research its origin story. Read the license text. Set up Linux environment (VM or lab system). Installed chosen software.	*Choice confirmed*
2	**Philosophy**	Written Part A1- the origin story. Taken notes on what problem the software solved and for whom.	*Draft A1*
3	**License & Ethics**	Written Part A2- license analysis. Written Part A3 - ethical reflection.	*Draft A2 + A3*
4	**Linux Footprint**	Installed software on Linux. Document directories, user, permissions. Written Part B.	*Draft Part B + screenshots*
5	**Scripts 1 & 2**	Written and tested Script 1 (System Identity) and Script 2 (Package Inspector). Added them to the report with brief explanations.	*Scripts 1-2 working*
6	**Scripts 3 & 4**	Written and tested Script 3 (Disk Auditor) and Script 4 (Log Analyzer). Tested with a real log file.	*Scripts 3-4 working*
7	**Ecosystem**	Written Part C- FOSS ecosystem map. Researched dependencies and community.	*Draft Part C*
8	**Comparison & Script 5**	Written Part D- comparison table and verdict. Written and tested Script 5 (Manifesto Generator).	*Draft Part D + Script 5*
9	**Report polish**	Compiled full report. Written introduction and conclusion. Added screenshots. Checked page count.	*Complete draft*
10	**Submission**	Final review. Submitted report + 5 .sh files (with GitHub repository link).	*Final submission*